Royaume du Maroc
Université Sultan Moulay Slimane
École Nationale des Sciences Appliquées
Béni Mellal
Transformation Digitale Industrielle

Spécialité : Transformation Digitale Industrielle
Module : Génie Logiciel

TP N° 1 : POO

Filière : **TDI**
Niveau : **2ème CI**

Effectuée par :

HAMMAOUI Fatima Ezzahra

Encadré(e) par :

Pr. BE.ELBAGHAZAOUI

Objectif du TP :

Ce travail pratique a pour objectif de maîtriser les concepts fondamentaux de la programmation orientée objet (POO) en Java, notamment l'encapsulation, l'héritage, l'abstraction et le polymorphisme, à travers la conception et l'implémentation de systèmes simples (gestion d'animaux, de véhicules et de comptes bancaires). Il vise également à développer la capacité à structurer un programme en classes, à réutiliser le code via l'héritage et à adapter le comportement des objets selon leur type.

A.U : 2025/2026

Exercice 1.1 – Programmation Orienté Objet (Code java)

Vous travaillez sur un système de gestion d'animaux dans un zoo. Chaque animal a un nom et une méthode pour produire un son spécifique. En plus de cela, certains animaux sont des mammifères, et d'autres des oiseaux. Vous devez compléter le programme en suivant les concepts POO et répondre aux questions ci-dessous.a

```java
// 1. Classe abstraite Animal
abstract class Animal {
    private String nom;
    public Animal(String nom) {
        this.nom = nom;
    }
    public String getNom() {
        return nom;
    }
    // Méthode abstraite à implémenter dans les sous-classes
    public abstract void faireDuBruit();
}
// 2. Classe Mammifere héritant d'Animal
class Mammifere extends Animal {
    public Mammifere(String nom) {
        super(nom);
    }
    @Override
    public void faireDuBruit() {
        System.out.println(getNom() + " grogne.");
    }
}
// 3. Classe Oiseau héritant d'Animal
class Oiseau extends Animal {
    public Oiseau(String nom) {
        super(nom);
    }
    @Override
    public void faireDuBruit() {
        System.out.println(getNom() + " chante.");
    }

    // Complétez cette méthode pour ajouter un comportement unique aux oiseaux
    // public void voler() {
    // }
}
// 4. Classe principale avec polymorphisme
public class Zoo {
    public static void main(String[] args) {
        Animal tigre = new Mammifere("Tigre");
        Animal perroquet = new Oiseau("Perroquet");
        tigre.faireDuBruit();     // Sortie attendue : "Tigre grogne."
        perroquet.faireDuBruit(); // Sortie attendue : "Perroquet chante."
        // Polymorphisme : la méthode faireDuBruit() se comporte différemment
        // selon l'instance d'Animal (tigre ou perroquet)
    }
}
```

Partie 1 : Définitions théoriques

Q1- Encapsulation : Expliquez ce qu'est l'encapsulation et comment elle est utilisée dans la classe Animal.

→L'encapsulation consiste à **cacher les données internes** d'une classe et y accéder via des méthodes.

utiliser comme ci :

- ✓ **private String nom;**
- ✓ **public String getNom();**
- ✓ **nom est privé**
- ✓ **accessible via getter protection des données**

Q2- Héritage : Décrivez ce qu'est l'héritage et identifiez les classes qui utilisent ce principe dans le code ci-dessus.
→L'Héritage permet à une classe de réutiliser les propriétés d'une autre.
utiliser comme ci :

- ✓ **class Mammifere extends Animal**
- ✓ **class Oiseau extends Animal**

Q3- Abstraction : Que signifie l'abstraction en POO ? Où est-elle mise en œuvre dans cet exercice ?
→L'abstraction consiste à définir un modèle sans implémentation complète.
utiliser comme ci :

- ✓ **abstract class Animal**
- ✓ **public abstract void faireDuBruit();**

Q4- Polymorphisme : Expliquez le polymorphisme et montrez comment il est utilisé dans la méthode main().
→Le polymorphisme permet à une même méthode d'avoir plusieurs comportements.
Exemple :

- ✓ **Animal tigre = new Mammifere("Tigre");**
- ✓ **Animal perroquet = new Oiseau("Perroquet");**
  - même type Animal mais le comportement est différent

Partie 2 : Compléter le code

Q5- Complétez la méthode voler() dans la classe Oiseau pour afficher "L'oiseau vole." quand elle est appelée.

```java
// class Oiseau complete:
class Oiseau extends Animal {
    public Oiseau(String nom) {
        super(nom);
    }

    @Override
    public void faireDuBruit() {
        System.out.println(getNom() + " chante.");
    }

    public void voler() {
        System.out.println(getNom() + " vole.");
    }
}
```

Q6- Modifiez le code dans la méthode main() pour que l'oiseau puisse aussi utiliser la méthode voler() (en supposant que l'objet perroquet est utilisé comme un Oiseau, et non un Animal).

```java
// 4. Classe principale avec polymorphisme
public class Zoo {
    Run | Debug
    public static void main(String[] args) {
        // Polymorphisme : la méthode faireDuBruit() se comporte différemment
        // selon l'instance d'Animal (tigre ou perroquet)

        Animal tigre = new Mammifere(nom: "Tigre");
        tigre.faireDuBruit();

        Oiseau perroquet = new Oiseau(nom: "Perroquet");
        perroquet.faireDuBruit();
        perroquet.voler();
    }
}
```

Partie 3 : Identification des mots-clés

7. Identifiez les mots-clés dans le code Java qui sont utilisés pour :

o L'héritage : extends

o La redéfinition de méthode : @Override

o L'abstraction : abstract

Exercice 1.2 - Programmation Orienté Objet (Code java)

Vous devez implémenter un système de gestion des véhicules pour une société de transport. Il existe différents types de véhicules, comme les voitures et les camions, qui partagent des caractéristiques communes mais ont également des comportements spécifiques. Vous allez utiliser les concepts POO pour modéliser ce système.

```java
// 1. Classe abstraite Véhicule
abstract class Vehicule {
    private String marque;
    private String modele;
    private int annee;
        public Vehicule(String marque, String modele, int annee) {
        this.marque = marque;
        this.modele = modele;
        this.annee = annee;
    }
        public String getMarque() {
        return marque;
    }
        public String getModele() {
        return modele;
    }
        public int getAnnee() {
        return annee;
    }
        // Méthode abstraite à implémenter dans les sous-classes
    public abstract void afficherDetails();
}
// 2. Classe Voiture qui hérite de Véhicule
class Voiture extends Vehicule {
    private int nombreDePortes;
        public Voiture(String marque, String modele, int annee, int nombreDePortes) {
        super(marque, modele, annee);
        this.nombreDePortes = nombreDePortes;
    }
        @Override
    public void afficherDetails() {
        System.out.println("Voiture: " + getMarque() + " " + getModele() + " (" + getAnnee() +
"), Portes: " + nombreDePortes);
    }
}
// 3. Classe Camion qui hérite de Véhicule
class Camion extends Vehicule {
    private double capaciteDeCharge;
        public Camion(String marque, String modele, int annee, double capaciteDeCharge) {
        super(marque, modele, annee);
        this.capaciteDeCharge = capaciteDeCharge;
    }

    @Override
    public void afficherDetails() {
```

```java
        System.out.println("Camion: " + getMarque() + " " + getModele() + " (" + getAnnee() + "),
Capacité: " + capaciteDeCharge + " tonnes");
    }
}
// 4. Classe principale
public class GestionTransport {
    public static void main(String[] args) {
        Vehicule maVoiture = new Voiture("Toyota", "Corolla", 2021, 4);
        Vehicule monCamion = new Camion("Volvo", "FMX", 2019, 12.5);
        maVoiture.afficherDetails(); // Voiture: Toyota Corolla (2021), Portes: 4
        monCamion.afficherDetails(); // Camion: Volvo FMX (2019), Capacité: 12.5 tonnes
                // Polymorphisme : la méthode afficherDetails() se comporte différemment
        // selon l'instance de Vehicule (Voiture ou Camion)
    }
}
```

Partie 1 : Théorie et concepts

1. Encapsulation : Identifiez dans le code où et comment le concept d'encapsulation est utilisé.

Attributs privés + getters : private String marque;

2. Héritage : Expliquez comment l'héritage est implémenté entre les classes Vehicule, Voiture et Camion.

- ✓ **class Voiture extends Vehicule**
- ✓ **class Camion extends Vehicule**

3. Polymorphisme : Expliquez comment le polymorphisme est utilisé dans la méthode main().

- ✓ **Vehicule v = new Voiture(...)**

4. Abstraction : Montrez où l'abstraction est utilisée dans ce code. Pourquoi Vehicule est-elle une classe abstraite ?

- ✓ **abstract class Vehicule :** empêche l'instanciation directe

Partie 2 : Compléter le code

5. **Ajoutez une méthode demarrer() dans la classe Vehicule qui est une méthode concrète et qui affiche "Le véhicule démarre." . Modifiez les classes Voiture et Camion pour qu'elles appellent cette méthode dans leurs propres méthodes afficherDetails().**

```java
// 1. Classe abstraite Véhicule
abstract class Vehicule {
 private String marque;
 private String modele;
 private int annee;
 public Vehicule(String marque, String modele, int annee) {
 this.marque = marque;
 this.modele = modele;
 this.annee = annee;
 }
 public String getMarque() {
 return marque;
 }
 public String getModele() {
 return modele;
 }
 public int getAnnee() {
 return annee;
 }
 // Méthode abstraite à implémenter dans les sous-classes
 public abstract void afficherDetails();
 public void demarrer() {
    System.out.println(x: "Le véhicule démarre.");
 }
```

```java
// 2. Classe Voiture qui hérite de Véhicule
class Voiture extends Vehicule {
 private int nombreDePortes;
 public Voiture(String marque, String modele, int annee, int nombreDePortes) {
 super(marque, modele, annee);
 this.nombreDePortes = nombreDePortes;
 }
//  @Override
//  public void afficherDetails() {
//  System.out.println("Voiture: " + getMarque() + " " + getModele() + " (" + getAnnee() +
// "), Portes: " + nombreDePortes);
//  }
 @Override
 public void afficherDetails() {
     demarrer();
     System.out.println("Voiture: " + getMarque() + " " + getModele() +
     " (" + getAnnee() + "), Portes: " + nombreDePortes);
 }
 }
```

```java
// 3. Classe Camion qui hérite de Véhicule
class Camion extends Vehicule {
 private double capaciteDeCharge;
 public Camion(String marque, String modele, int annee, double capaciteDeCharge) {
 super(marque, modele, annee);
 this.capaciteDeCharge = capaciteDeCharge;
 }
//  @Override
//  public void afficherDetails() {System.out.println("Camion: " + getMarque() + " " + getModele() + " (" + getAn
//  }
 @Override
 public void afficherDetails() {
    demarrer();
    System.out.println("Camion: " + getMarque() + " " + getModele() +
    " (" + getAnnee() + "), Capacité: " + capaciteDeCharge + " tonnes");
 }
}
```

6. Créez une nouvelle classe Moto qui hérite de Vehicule, avec un attribut typeDeGuidon. Assurez-vous d'implémenter la méthode afficherDetails() pour cette classe.

```java
// Q6: class Moto :
class Moto extends Vehicule {

    private String typeDeGuidon;

    public Moto(String marque, String modele, int annee, String typeDeGuidon) {
        super(marque, modele, annee);
        this.typeDeGuidon = typeDeGuidon;
    }

    @Override
    public void afficherDetails() {
        demarrer();
        System.out.println("Moto: " + getMarque() + " " + getModele() +
        " (" + getAnnee() + "), Guidon: " + typeDeGuidon);
    }
}
```

Partie 3 : Identification des mots-clés

7. Identifiez les mots-clés dans le code utilisés pour :
o La redéfinition de méthode : @Override
o L'héritage : extends
o L'appel au constructeur parent : super

Exercice 1.3 - Programmation Orienté Objet (implementation java)

Vous devez créer un programme qui gère différents types de comptes bancaires : CompteCourant et CompteEpargne. Chaque compte a un numéro, un solde, et des méthodes pour déposer et retirer de l'argent. Le CompteEpargne a en plus une méthode pour calculer les intérêts. Instructions :

1. Créez une classe de base CompteBancaire avec les attributs et méthodes suivants :

o Attributs :
numeroCompte (String) : numéro de compte
solde (double) : solde du compte

o Méthodes :
deposer(double montant) : méthode pour déposer de l'argent sur le compte.
retirer(double montant) : méthode pour retirer de l'argent du compte avec une vérification du solde.

```java
class CompteBancaire {

    protected String numeroCompte;
    protected double solde;

    public CompteBancaire(String numeroCompte, double solde) {
        this.numeroCompte = numeroCompte;
        this.solde = solde;
    }

    public void deposer(double montant) {
        solde += montant;
    }

    public void retirer(double montant) {
        if (montant <= solde) {
            solde -= montant;
        } else {
            System.out.println(x: "Solde insuffisant !");
        }
    }

    public void afficherSolde() {
        System.out.println("Solde: " + solde);
    }
}
```

2. Créez une classe dérivée CompteEpargne qui hérite de CompteBancaire et ajoute une méthode :

- calculerInterets(double taux) : méthode pour calculer et ajouter des intérêts au solde selon un taux donné.

```
class CompteEpargne extends CompteBancaire {

    public CompteEpargne(String numeroCompte, double solde) {
        super(numeroCompte, solde);
    }

    public void calculerInterets(double taux) {
        solde += solde * taux;
    }
}

class CompteCourant extends CompteBancaire {

    public CompteCourant(String numeroCompte, double solde) {
        super(numeroCompte, solde);
    }
}
```

3. Créez une classe principale Banque avec la méthode main() qui crée des objets de type CompteCourant et CompteEpargne, et qui teste les fonctionnalités.

```
public class Banque {

    Run | Debug
    public static void main(String[] args) {

        CompteCourant c1 = new CompteCourant(numeroCompte: "C001", solde: 1000);
        CompteEpargne c2 = new CompteEpargne(numeroCompte: "E001", solde: 2000);

        c1.deposer(montant: 500);
        c1.retirer(montant: 300);
        c1.afficherSolde();

        c2.calculerInterets(taux: 0.05);
        c2.afficherSolde();
    }
}
```